

My 2026 Automation Stack Future or Finished.

the 8 verdicts from the reel, in full — and the 5 tools I actually run every single day

```
$ zapier upgrade --plan "pay per task, forever"  
$ pip install langchain --wrappers 47  
$ chatgpt ask --same-question --again --tomorrow
```

Everything in that terminal is **finished**. Not because the tools are bad — because their *business model* is. Per-task pricing and wrapper layers can't survive a world where an AI agent does the task for free. In the reel I gave you the verdicts; this guide is the full breakdown I promised: **why** each tool lives or dies, the exact 5-tool stack I run at ZeroHands every day, what each one replaces, what it roughly costs, the setup command, and the pitfall that catches everyone the first time.

The verdict table

Screenshot this. Then argue with me in the comments.

TOOL	VERDICT	ONE-LINE REASON
ChatGPT (chat window)	FINISHED	Asking AI for help is the calculator era — the answer dies in the chat, nothing compounds.
ChatGPT Agent mode	FUTURE	The moment it stops answering and starts <i>doing</i> , it's a different product.
Zapier	FINISHED	Per-task pricing dies when an agent runs the same task for free.
n8n	FUTURE	Self-hosted, free, and it runs agents — not just triggers.
LangChain	FINISHED	A wrapper layer the models absorbed — the glue became a feature.
Make	FINISHED	Same per-operation meter as Zapier. Same meter, same fate.
HeyGen	FUTURE	Film once, publish forever — it makes the camera optional.
Claude Code	FUTURE	The terminal that builds everything else on this list.
AI agents	FUTURE	Not a tool — the category everything above is being rebuilt around.

Notice the pattern: everything **finished** charges you per task or wraps someone else's model. Everything **future** either *is* an agent, *runs* agents, or produces an asset agents can't (your face, your voice). Keep that filter — it'll outlive this list.

The stack I run daily

Five tools. Together they replace a developer, an ops hire, a videographer, an editor, and four subscriptions to video-gen apps. Here's each one, honestly.

1 Claude Code — the brain

REPLACES · A DEVELOPER + EVERY "NO-CODE" TOOL AT ONCE

Claude Code is Claude running in your terminal with permission to read files, write code, and run commands. It doesn't suggest an automation — it **builds and executes** it. My entire content pipeline (scripts, video edits, ffmpeg compositing, API calls) runs from one-line commands that Claude Code wrote and now operates.

The compounding trick is **skills**: instruction files saved at `.claude/skills/<name>/SKILL.md` inside a project. Claude auto-loads them whenever they're relevant — so it permanently knows your tools,

your voice, your commands. You explain the job once, then every future session starts already-trained.

```
# install (macOS / Linux), then run it inside any project folder
npm install -g @anthropic-ai/claude-code
cd ~/my-project && claude
```

PROMPT — YOUR FIRST REAL AUTOMATION (PASTE AS-IS)

Look at this project folder. Find the most repetitive manual task you can see evidence of (file naming, format conversion, copy-paste between files, repeated commands). Build a single script that automates it end to end, save it as `automate.sh` (or `automate.py`), run it once to prove it works, and show me the before/after. Then write a one-line command I can run any time to repeat it.

Cost: Claude Pro (~\$20/mo) covers real daily use; heavier builders move to Max. Compare that to one hour of a freelance developer and the math ends the conversation.

Setup tip: create one skill file per repeated job (script writer, report builder, video compositor). Prompts don't compound — skills do. If you're re-explaining something in every chat, that explanation belongs in a `SKILL.md`.

Pitfall: treating it like a chatbot. If you ask questions, you get answers. Give it a **goal and permission** ("build it, run it, show me the output") and it becomes an employee. The verdict table above is literally this distinction — chat is finished, agents are the future.

2 n8n — the workflow engine

REPLACES · ZAPIER + MAKE (AND THEIR PER-TASK BILLS)

n8n is the automation canvas that survived my own verdict table, for one structural reason: you can **self-host it for free**, and it has native AI-agent nodes. Zapier charges you every time a task runs; n8n on your own machine runs 10 or 10,000 executions for the same price: zero. It's where my always-on workflows live — the things that should run at 3am without me: lead capture to CRM, content-posting schedules, inbox triage, report generation.

```
# self-host n8n in one command (needs Docker installed — docker.com, free)
docker run -it --rm -p 5678:5678 -v n8n_data:/home/node/.n8n
docker.n8n.io/n8nio/n8n

# then open http://localhost:5678 in your browser — that's it
```

PROMPT — LET CLAUDE CODE BUILD THE WORKFLOW FOR YOU

I'm running n8n locally at <http://localhost:5678>. Design a workflow for this task: [describe ONE repetitive task, e.g. "when a new row lands in my Google Sheet of leads, draft a personalised intro email and save it to Gmail drafts"]. Give me the complete workflow as importable n8n JSON, list every credential I need to connect and where to click to add each one, then walk me through testing it with one sample row.

Cost: \$0 self-hosted (their cloud plan starts ~€24/mo if you'd rather not run Docker). The real cost is the one evening it takes to ship your first workflow.

Setup tip: pick ONE task that happens at least daily and annoys you every time. One boring workflow that runs every day beats a spectacular one that never ships.

Pitfall: building ten half-finished workflows. The graveyard of automation is full of 80%-done canvases. Finish one, watch it run unattended for a week, *then* start the second.

3 HeyGen — the face

REPLACES · THE CAMERA, THE STUDIO, THE RESHOOT

HeyGen trains a talking avatar from roughly **2 minutes of real footage**. Type a script, get yourself on camera — same face, same voice. I filmed one session and trained **17 looks across 5 outfits**; every reel on my feed since then was typed, not filmed. This is the "produce an asset" side of the future column: agents can write and edit, but your face is yours.

PROMPT — SCRIPT IT BEFORE YOU RENDER IT (PASTE INTO CLAUDE CODE)

Write a 45-60 second talking-head reel script on [topic]. Rules: hook in the first 3 seconds that opens a loop, one core idea only, spoken 8th-grade English, short sentences, name one specific tool or number, end with "Comment [KEYWORD] and I'll send you [thing]". 110-150 words. Plain text only — I'm pasting it straight into a HeyGen avatar render.

Cost: creator plans start around the price of a lunch per month; on API plans think roughly **one credit ≈ one minute of avatar video**. A weekly posting schedule burns a handful of credits — trivially cheap next to filming.

Setup tip: at your one filming session, capture **multiple looks** — 2–3 outfits, 2 settings, front and side angles, ~2 minutes of natural talking each. Rotating looks is what makes the feed look alive instead of

like one recording session on repeat.

Pitfall: the ~5,000 character input limit, and rendering unapproved drafts. Scripts are free; renders cost credits. Lock the script first, render once.

4 ElevenLabs — the voice + the timing

REPLACES · THE VIDEO EDITOR'S ENTIRE SYNC JOB

Two jobs. **Voice:** the best voice cloning and text-to-speech available — if an avatar's built-in voice sounds flat, an ElevenLabs clone of your real voice fixes it. **Timing:** their **Scribe** speech-to-text returns a timestamp for *every single word* in a video. Once you know exactly when each word lands, everything an editor used to eyeball becomes math — captions light up on the spoken word, graphics pop at the name-drop, cuts land at sentence ends.

```
# word-level timestamps for any video — one curl (key from elevenlabs.io → API keys)
curl -s https://api.elevenlabs.io/v1/speech-to-text \
  -H "xi-api-key: YOUR_API_KEY" \
  -F model_id=scribe_v1 \
  -F file=@my_video.mp4 > words.json

# words.json now holds {"word":"step","start":0.12,"end":0.31}, for every word
```

Cost: Scribe runs about **\$0.013 for a 2-minute reel**. One cent replaces the sync work of a human editor — the single best money in this entire stack. Voice plans start free; the paid tiers begin around \$5/mo.

Setup tip: use Scribe over free local Whisper. I ran Whisper first — it mishears every proper noun ("Claude" becomes "clot", "n8n" becomes "n 8 n"), which breaks captions on exactly the words that matter. For a cent per reel this isn't a decision.

Pitfall: transcribing the raw render instead of the final trimmed video. If you trim after transcribing, every timestamp is offset and every caption fires early. Trim first, transcribe last.

5 Kie.ai — the generation backend

REPLACES · FOUR SEPARATE SUBSCRIPTIONS TO VIDEO/IMAGE APPS

Kie.ai is one API key that routes to the frontier generation models — Kling for cinematic video, nano-banana for images, ElevenLabs sound effects, and more — **pay-per-generation** instead of a monthly plan per app. Notice it survives my own "per-task pricing is finished" rule, because it meters GPU

output (which genuinely costs money per run), not a workflow step an agent could do for free. All my AI b-roll and generated imagery flows through it.

```
# generate a 5-second vertical b-roll clip (key from kie.ai dashboard)
curl -s -X POST https://api.kie.ai/api/v1/jobs/createTask \
  -H "Authorization: Bearer YOUR_KIE_KEY" -H "Content-Type: application/json" \
  -d '{"model":"kling-2.6/text-to-video","callbackUrl":"","input":{"
    "prompt":"dark server room, slow dolly forward, red accent light, cinematic,
no text, no watermarks, no faces",
    "duration":5,"aspect_ratio":"9:16"}}'
```



```
# response gives a taskId — poll until the video URL appears:
curl -s https://api.kie.ai/api/v1/jobs/task/TASK_ID -H "Authorization: Bearer
YOUR_KIE_KEY"
```

Cost: credits, cents per generation — a 5-second video clip costs less than a coffee, an image costs less than the sugar in it. You top up when you produce, and pay nothing when you don't.

Setup tip: end every video prompt with *"no text, no watermarks, no subtitles, no captions"* — generation models love inventing fake UI text, and one watermarked clip ruins an otherwise clean edit.

Pitfall: a 200 OK response means the task was **created**, not completed. Always poll the task URL until the output appears — treating 200 as "done" is the #1 beginner integration bug.

Why the finished column is finished

Per-task pricing is a bet against your own growth. Zapier and Make charge you every time an automation runs — which means the better your automation works, the bigger your bill. That model made sense when connecting two apps required their engineering. It stops making sense the day an agent can write the same integration as a script on your machine and run it a million times for free. You're no longer paying for the task; you're paying rent on not knowing it's now free.

Wrappers die when the model absorbs them. LangChain's whole value was glue — chaining prompts, managing memory, calling tools — at a moment when raw models couldn't do any of that. Every one of those features now ships inside the models and inside agent products like Claude Code. When the platform absorbs your layer, you don't compete with the platform; you become a changelog entry. The same logic applies to every "AI-powered" app that is a thin prompt over someone else's model.

What survives is assets and engines. Look at the future column again: an agent (Claude Code), an engine you own and self-host (n8n), and asset factories (HeyGen, ElevenLabs, Kie.ai) that produce

things — your face, your voice, footage — that agents consume but can't conjure. That's the durable shape of a 2026 stack: **one brain, one engine you own, and assets nobody can meter.**

Quickstart — your first week

Start with 3 of the 5. In this order.

1. **Day 1–2: Claude Code.** Install it, open your messiest project folder, and run the "first real automation" prompt from Tool 1. Ship one working script. This is the brain — everything else gets easier once it's in place.
2. **Day 3–4: n8n.** Run the Docker one-liner, then have Claude Code design your first workflow (the Tool 2 prompt). One repetitive daily task, fully finished, running unattended.
3. **Day 5–7: ElevenLabs.** Get an API key, run one Scribe transcription on any video you have, and open the words JSON. The moment you see per-word timestamps, you'll understand how every "how is this edited so fast" video on your feed is actually made.

```
week 1: brain (Claude Code) → engine (n8n) → timing (ElevenLabs)
week 2+: add the face (HeyGen) and the footage (Kie.ai)
        only when you're ready to publish — not before.
```

The 5 mistakes that stall beginners

- **Buying the whole stack on day one** — five subscriptions and zero shipped automations is the most expensive way to feel productive. Three tools, one week, one thing running.
- **Staying in the chat window** — if your AI use is still question → answer → copy-paste, you're in the finished column of your own life. Give the agent the goal and the keyboard.
- **Paying per task out of habit** — every Zapier zap you keep is a subscription to not learning n8n. Migrate the top 3 zaps first; the rest follows.
- **Automating the interesting task instead of the boring one** — boring + daily beats impressive + occasional, every time. The ROI is in the repetition, not the demo.
- **Never letting it run unattended** — an automation you babysit is just a slower way of doing it yourself. The test is a full week of it running while you don't watch.

Want this done FOR you? — ZeroHands

If you'd rather have **ZeroHands** build and run this whole stack for your business — the agent setup, the workflows, the content system, all of it — book a free 20-minute call. You show up once; we handle everything after.

Book a free call →

calendly.com/anand_kaliappan/call · zerohands.co

This is the exact stack I run at ZeroHands

One brain, one engine, three asset factories — and a feed that publishes daily without a camera. I share the real builds — screens, prompts, and the misses too — one automation at a time.

Follow **@automatewithanand** on Instagram · Subscribe on YouTube · Comment **STACK** on the reel if you want the next guide first.